



QUESTION BANK- DATA STRUCTURES

SHORT ANSWERS (Module 1, Module 2, Module 3)

1. Define Data Structure.

A data structure is a method of storing and organizing data in a main memory so it can be used efficiently.

Examples: arrays, linked lists, stacks, queues, trees.

They improve data access and management.

2. Define Recursion.

Recursion is a technique where a function calls itself repeatedly until a base condition stops the calls. It solves problems by dividing them into smaller versions of the same problem.

Examples: factorial, Fibonacci, Towers of Hanoi.

3. Why Data Structures needed?

The purpose of a data structure is not just to store data, but to organize it in such a way that, in the future, we can efficiently retrieve and easily manipulate the data. They reduce time and memory usage and make searching, sorting, and processing faster. Without data structures, even simple operations become slow on large data.

4. Difference between Time Complexity and Space Complexity.

Time complexity measures how much **time** an algorithm takes based on input size.

Space complexity measures how much **memory** an algorithm uses.

Both help evaluate and compare algorithm efficiency.

5. What is Algorithm Efficiency?

- Algorithm efficiency describes how fast or how much memory an algorithm uses as input size increases.
- It is measured using time complexity and space complexity.
- Efficient algorithms run faster and consume fewer resources.

6. What is Big-O notation?

Big-O notation represents the **worst-case time complexity** of an algorithm.

It gives the upper limit of the running time as input size increases.

Example: Linear Search worst-case $\rightarrow O(n)$.

7. What is Bubble Sort Complexity in worst case.

- In the worst case, Bubble Sort takes **$O(n^2)$** time.



- This occurs when the array is in reverse order.
- Maximum comparisons and swaps happen.

8. Define Sorting.

- Sorting is arranging data in increasing or decreasing order.
- It makes searching and accessing data faster.

Example: Bubble Sort, Insertion Sort, Merge Sort.

9. What is a Sorting Algorithm?

A sorting algorithm arranges elements of a list in ascending or descending order. It improves searching efficiency and organizes data.

Example: Bubble Sort, Merge Sort, Quick Sort.

10. What is the complexity of Binary Search?

- Binary Search has a time complexity of $O(\log n)$.
 - This is because the input list is divided into half each time.
- It works only on sorted arrays.

11. What is Merge Sort Complexity?

- Merge Sort has a time complexity of $O(n \log n)$ in best, worst, and average cases.
- This is because it divides the array repeatedly and merges sorted halves.
- It is faster than simple sorting algorithms like Bubble Sort.

12. Define Multidimensional Array.

A multidimensional array is an array that contains more than one dimension, meaning it is an array of arrays. It is used to store data in the form of rows and columns (or higher dimensions) under a single variable name. Most commonly, a two-dimensional array (2D array) is used, which looks like a table or matrix.

13. Define Linked List and how is it different from an array.

A linked list is a structure where each element (node) has data and a pointer to next node. It uses non-contiguous memory, unlike arrays which use continuous memory. Linked lists grow dynamically and allow easy insertions and deletions.

14. What is a Hash Table?

A hash table stores data in key–value format using a hash function. It provides very fast access (average $O(1)$ time). Used in databases, compilers, and dictionaries.

15. Define Singly Linked List.

A singly linked list is a data structure where each node contains data and a pointer to the next node. It allows sequential access from the first node to the last node. The last node points to NULL.



LONG ANSWERS (Module 1, Module 2, Module 3)

1. Explain the Towers of Hanoi problem for 3 disks. Write a code to implement using recursion.

The Towers of Hanoi is a mathematical puzzle used to explain the concept of recursion.

It consists of three pillars (rods):

- Source
- Auxiliary
- Destination

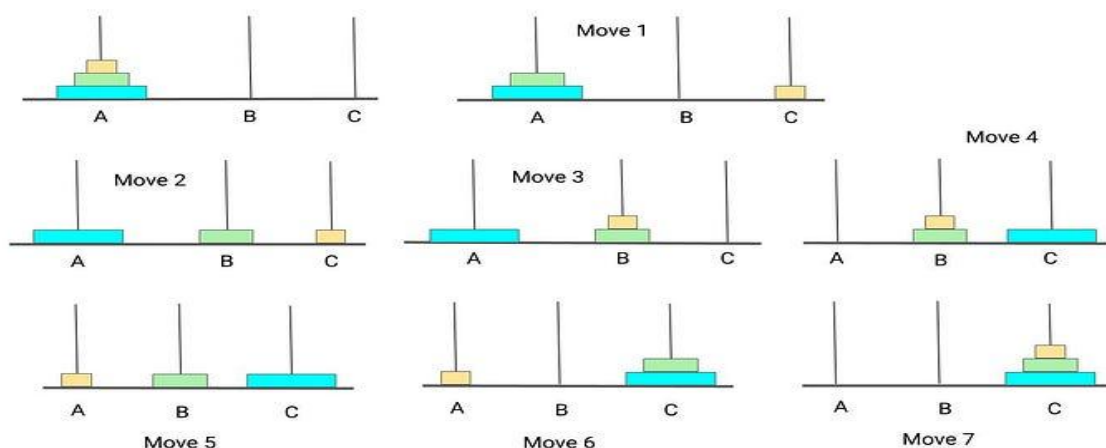
and three disks of different sizes placed on the Source rod in ascending order of size (smallest at the top, largest at the bottom).

Rules of the Problem

1. Only one disk can be moved at a time.
2. Only the top disk of any rod can be moved.
3. A larger disk cannot be placed on a smaller disk.

Objective

Move all three disks from the Source rod to the Destination rod using the Auxiliary rod, without changing the order of disks.



Steps for 3 Disks

1. Move disk 1 from Source → Destination



2. Move disk 2 from Source → Auxiliary
3. Move disk 1 from Destination → Auxiliary
4. Move disk 3 from Source → Destination
5. Move disk 1 from Auxiliary → Source
6. Move disk 2 from Auxiliary → Destination
7. Move disk 1 from Source → Destination

Total moves required = $2^3 - 1 = 7$ moves

Program:

```
def TOH(n,source,auxillary,destination):  
    if n==0:  
        return  
    else:  
        TOH(n-1,source,destination,auxillary)  
        print(f'move disk {n} from {source}to{destination}')  
        TOH(n-1,auxillary,source,destination)  
  
n=3  
TOH(n,'A','B','C')
```

2. Apply the Sliding Window technique to find maximum sum of a subarray of size $k = 3$ for $A = [2, 1, 5, 1, 3, 2]$.

Sliding Window is used when:

- We need sum of subarrays
- Window (group size) is fixed

Here $k = 3$

Step-by-Step Calculation

Array:

[2, 1, 5, 1, 3, 2]

1) First window (indexes 0–2)

$$2 + 1 + 5 = 8$$

2) Slide window by 1 position



Remove element leaving window $\rightarrow 2$

Add new element entering window $\rightarrow 1$

New sum $= 8 - 2 + 1 = 7$

3) Next window

Remove 1, add 3

New sum $= 7 - 1 + 3 = 9$

4) Next window

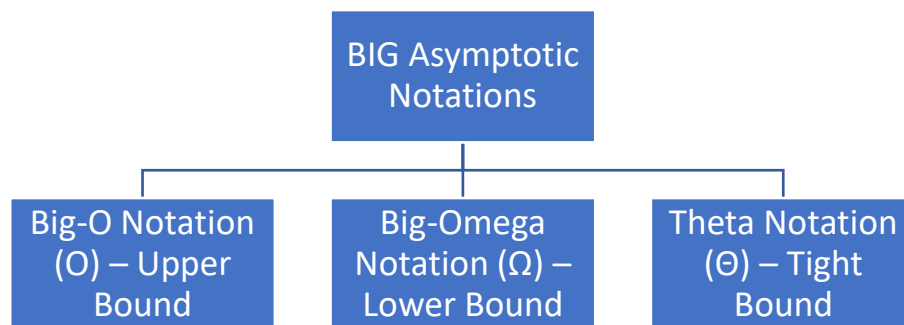
Remove 5, add 2

New sum $= 9 - 5 + 2 = 6$

Maximum sum = 9

3.Explain the different types of Asymptotic Notations (O , Ω , Θ).

Asymptotic notations are used to analyze the time complexity and space complexity of algorithms. They help us understand how an algorithm behaves when the input size becomes very large. The three most commonly used asymptotic notations are:



1.Big-O Notation (O)

- Big-O notation represents the upper bound of an algorithm.
- It describes the worst-case time complexity.
- It shows the maximum time an algorithm may take for a given input size.
- Big-O is used to compare algorithms based on efficiency.
- It ignores constants and lower-order terms.
- It is the most commonly used asymptotic notation.

Examples:



- Linear Search – $O(n)$
- Bubble Sort – $O(n^2)$

2. Big-Omega Notation (Ω)

- Big-Omega notation represents the lower bound of an algorithm.
- It describes the best-case time complexity.
- It shows the minimum time required by an algorithm.
- It helps to understand the fastest possible execution.
- Used when analyzing best-case scenarios.

Examples:

- Linear Search (element found first) – $\Omega(1)$
- Bubble Sort (already sorted) – $\Omega(n)$

3. Big-Theta Notation (Θ)

- Big-Theta notation represents the tight bound of an algorithm.
- It describes the exact growth rate of an algorithm.
- It gives both upper and lower bounds.
- Used when best-case and worst-case complexities are same.
- Provides a precise analysis of algorithm performance.

Examples:

- Merge Sort – $\Theta(n \log n)$
- Single loop – $\Theta(n)$

4. Construct Prefix Sum for $A = [2, 4, 6, 8, 10]$ and find sum from 1 to 3.

Given Array:

$A = [2, 4, 6, 8, 10]$

Prefix Sum Construction

Prefix sum is the cumulative sum of array elements.

$\text{prefix}[0] = 2$
 $\text{prefix}[1] = 2 + 4 = 6$
 $\text{prefix}[2] = 6 + 6 = 12$
 $\text{prefix}[3] = 12 + 8 = 20$
 $\text{prefix}[4] = 20 + 10 = 30$



Prefix Sum Array = [2, 6, 12, 20, 30]

Finding Sum from Index 1 to 3

Formula used:

$$\text{Sum}(l \text{ to } r) = \text{prefix}[r] - \text{prefix}[l - 1]$$

$$\text{Sum} = \text{prefix}[3] - \text{prefix}[0]$$

$$\text{Sum} = 20 - 2$$

$$\text{Sum} = 18$$

- Prefix Sum Array = [2, 6, 12, 20, 30]
- Sum of elements from index 1 to 3 = 18

#Program to find the range and prefix sum

```
arr=[10,20,30,40,50]
prefix_sum=[0]*len(arr)
prefix_sum[0]=arr[0]
for i in range(1,len(arr)):
    prefix_sum[i]=prefix_sum[i-1]+arr[i]
print("original array",arr)
print("prefix sum array",prefix_sum)
#To find the range sum
l=1
r=3
if l==0:
    range_sum=prefix_sum[r]
else:
    range_sum = prefix_sum[r]-prefix_sum[l-1]
print("rangesum is:",range_sum)
```



5. Write a program to perform sorting using bubble sort.

```
def bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(0, n - i - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]  
  
# Example  
arr = [5, 3, 1, 4, 2]  
bubble_sort(arr)  
print(arr)
```

Output

[1, 2, 3, 4, 5]

6. What is Recursion? Explain different Types of Recursion. Write a Program to find the factorial of a number using recursion.

Recursion is a programming technique in which a **function calls itself** to solve a problem. In recursion, a problem is divided into **smaller sub-problems**, and the function keeps calling itself until a stopping condition is reached. Recursion is mainly used in problems that can be defined in terms of **smaller versions of the same problem**, such as factorial, Fibonacci series, and Towers of Hanoi.

Components of Recursion

1. Base Case:

The **base case** is the condition that **stops the recursion**. It defines the smallest problem that can be solved directly without further function calls.

2. Function Call (Recursive Call)

The **recursive call** is the statement where the function **calls itself** with a smaller or modified input. Each recursive call works on a **reduced version of the problem** and waits for the result from the next call.

Example:



```
return n * factorial(n - 1)
```

Types Of Recursion

There are two main types of recursions:

- Direct Recursion
- Indirect Recursion

Direct Recursion: Direct recursion happens when a function calls itself directly inside its own body.

```
def recur(n):  
    if n == 0:      # base case  
        return  
    print(n)  
    recur(n - 1)    # function calls itself
```

Indirect Recursion::

If a function calls another function, and that second function calls back the first function, this cycle is called indirect recursion.

```
def recurA():  
    # function recurA() calling recurB()  
    recurB()  
def recurB():  
    # function recurB() calling recurA()  
    recurA()
```



Example: Factorial

```
def factorial(n):  
    if n == 0:          # Base case  
        return 1  
    else:  
        return n * factorial(n - 1)  # Recursive call (change of condition)  
  
# Function call  
result = factorial(5)  
print(result)
```



OUTPUT

120

7. Explain Bubble Sort algorithm with an example .show the steps involved in sorting the array?

Bubble Sort is a simple sorting algorithm that repeatedly compares adjacent elements in a list and swaps them if they are in the wrong order. This process continues until the entire list is sorted. After each pass, the largest element “bubbles up” to the end of the list..

Steps

- Step 1: Compare first two elements
- Step 2: Swap if wrong order
- Step 3: Move to next pair
- Step 4: Repeat passes until sorted

Example

Working of Bubble Sort

Suppose we are trying to sort the elements in **ascending order** of Array= [-2,45,0,11,-9]

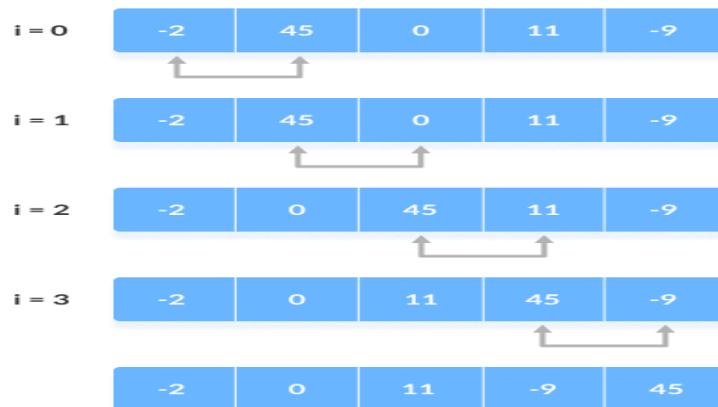
1. First Iteration (Compare and Swap)

1. Starting from the first index, compare the first and the second elements.



2. If the first element is greater than the second element, they are swapped.
3. Now, compare the second and the third elements. Swap them if they are not in order.
4. The above process goes on until the last element.

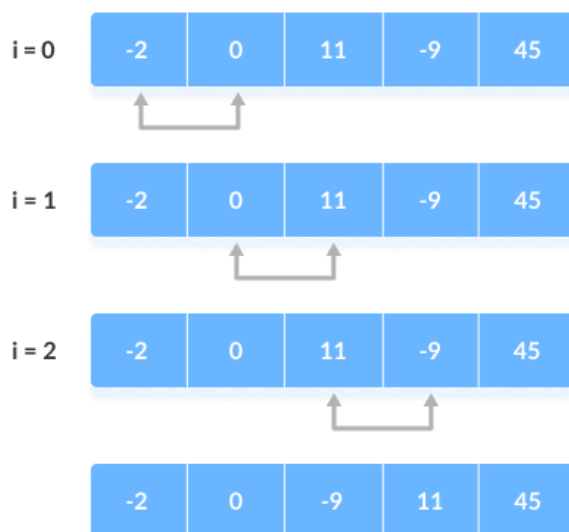
step = 0



2. Remaining Iteration

The same process goes on for the remaining iterations. After each iteration, the largest element among the unsorted elements is placed at the end.

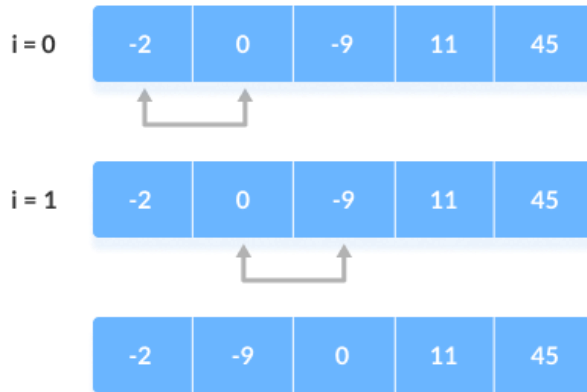
step = 1





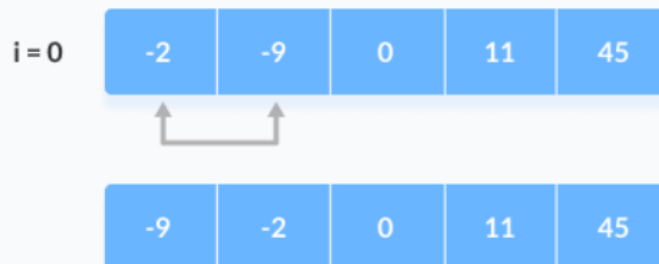
In each iteration, the comparison takes place up to the last unsorted element

step = 2



The array is sorted when all the unsorted elements are placed at their correct positions.

step = 3



8. Write a program to perform Binary Search on an array?

Binary Search works only on **sorted lists**. It divides the list into two halves repeatedly.

Steps

1. Find mid index
2. Compare mid element with key
3. If key < mid → search left half



4. If $\text{key} > \text{mid} \rightarrow$ search right half
5. Repeat until element is found or range ends

Program

```
def binary_search(arr, key):  
    low=0  
    high = len(arr)-1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == key:  
            return mid  
        elif key < arr[mid]:  
            high = mid - 1  
        else:  
            low = mid + 1  
    return -1  
arr = [10, 20, 30, 40, 50]  
print("Found at index:", binary_search(arr, 30))
```

OUTPUT

Found at index: 2

9. Write a program to perform Linear Search on an array.

Linear Search checks every element until the target is found.

Program:

```
def linear_search(arr, key):  
    for i in range(len(arr)):  
        if arr[i] == key:  
            return i  
    return -1
```



```
arr = [10, 20, 30, 40, 50]
```

```
key = 30
```

```
pos = linear_search(arr, key)
```

```
print("Element found at index:", pos)
```

OUTPUT

Element found at index: 2

10. Apply Selection Sort to sort the following array in ascending order: A = [85, 60, 25, 10, 40, 15].

Selection Sort — Steps

1. Read the elements into the array.
2. Start from the first position of the array.
3. Assume the first element is the smallest.
4. Compare this element with all remaining elements.
5. If a smaller element is found, note its position.
6. After completing comparisons, swap the smallest element with the current position.
7. Move to the next position and repeat the steps.
8. Continue until all elements are arranged in ascending order.

Given Array:

A = [85, 60, 25, 10, 40, 15]

Pass 1 (i = 1)

Minimum element in A[1 to 6] = **10**

Swap A[1] and A[4]

[10 60 25 85 40 15]

Pass 2 (i = 2)

Minimum element in A[2 to 6] = **15**

Swap A[2] and A[6]

[10 15 25 85 40 60]

Pass 3 (i = 3)



Minimum element in A[3 to 6] = **25**

Already in correct position (No swap)

[10 15 25 85 40 60]

Pass 4 (i = 4)

Minimum element in A[4 to 6] = **40**

Swap A[4] and A[5]

[10 15 25 40 85 60]

Pass 5 (i = 5)

Minimum element in A[5 to 6] = **60**

Swap A[5] and A[6]

[10 15 25 40 60 85]

Final Sorted Array:

[10, 15, 25, 40, 60, 85]

11. Apply Insertion Sort to A = [8, 3, 5, 2, 9]

Insertion Sort builds the final sorted array one element at a time.

Step-by-Step:

given Array:

A = [8, 3, 5, 2, 9]

Step 1: First element is considered sorted.

Sorted part: [8]

Unsorted part: [3, 5, 2, 9]

Step 2: Insert 3 into the sorted part.

Sorted part: [3, 8]

Unsorted part: [5, 2, 9]

Step 3: Insert 5 into the sorted part.

Sorted part: [3, 5, 8]

Unsorted part: [2, 9]



Step 4: Insert 2 into the sorted part.

Sorted part: [2, 3, 5, 8]

Unsorted part: [9]

Step 5

Insert 9 into the sorted part.

Sorted part: [2, 3, 5, 8, 9]

Unsorted part: []

The entire array becomes sorted as: [2, 3, 5, 8, 9]

12. Explain Merge Sort with example.

Merge Sort is a Divide and Conquer sorting algorithm. It divides the given array into smaller subarrays, sorts them recursively, and then merges the sorted subarrays using the two-pointer technique.

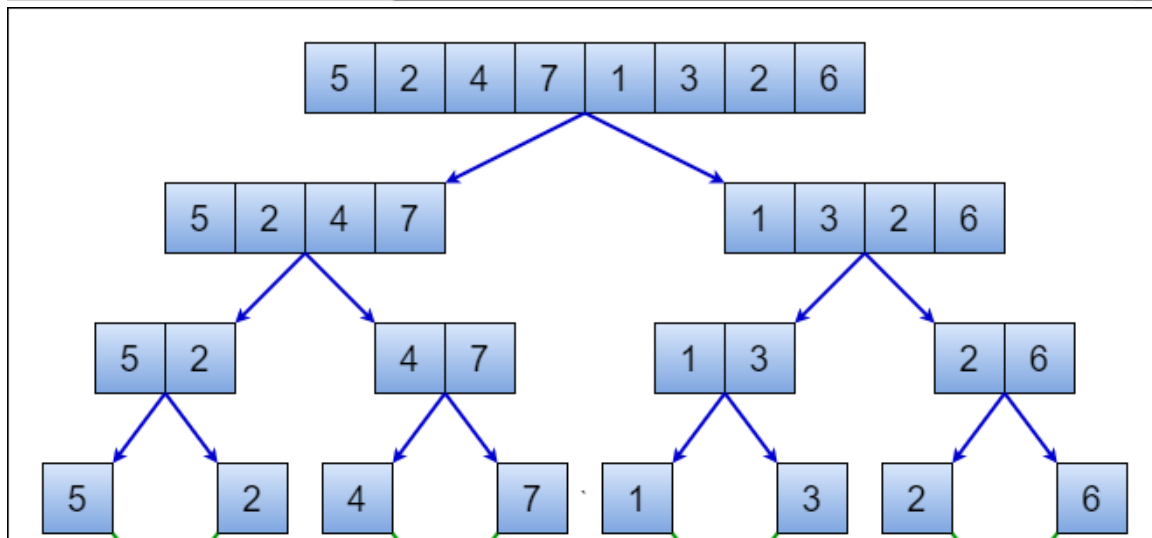
Algorithm Idea

1. Divide the array into two halves.
2. Recursively divide each half until each subarray has one element.
3. Merge the subarrays in sorted order using **two pointers**.
4. Continue merging until the entire array is sorted.

Example Array: [5, 2, 4, 7, 1, 3, 2, 6]

Phase 1: Divide Phase :

The array is recursively divided into halves until each subarray contains only one element.



Phase 2: Merge Phase: Now merging starts using the two-pointer concept

Key Two-Pointer Logic For merging two sorted array:

- Two pointers compare **one element from each subarray**
- Smaller element is copied to the result
- Pointer of the copied element is moved
- Remaining elements are copied after one array finishes

Step 1: Merge Single-Element Subarrays

$[5] + [2] \rightarrow [2\ 5]$

Compare 5 and 2. Since 2 is smaller, place 2 first and then copy 5.

Repeat same thing for all sub arrays

$[4] + [7] \rightarrow [4\ 7]$

$[1] + [3] \rightarrow [1\ 3]$

$[2] + [6] \rightarrow [2\ 6]$

Step 2: Merge Two-Element Sorted arrays



Left = [2, 5] Right = [4, 7]

[2 5] + [4 7] → [2 4 5 7]

- Compare elements from both subarrays using two pointers and place them in sorted order.

<i>Left[i]</i>	<i>Right[j]</i>	<i>Smaller</i>	<i>Result</i>
2	4	2	[2]
5	4	4	[2, 4]
5	7	5	[2, 4, 5]
—	7	copy remaining	[2, 4, 5, 7]

❖ [2,4,5,7]

❖ Repeat same thing all sub arrays

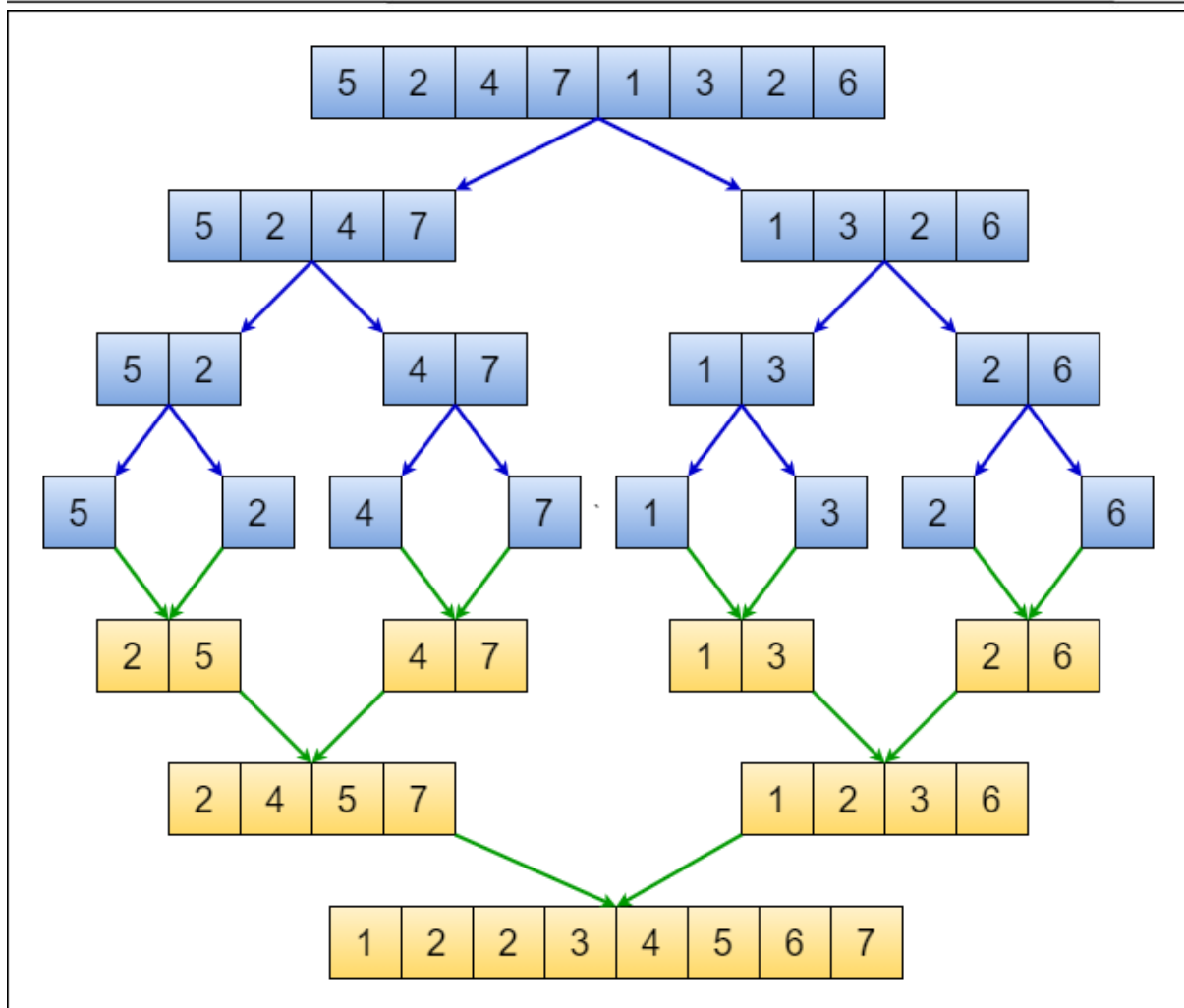
[1 3] + [2 6] → [1 2 3 6]

Compare elements from both subarrays using two pointers and merge them in ascending order.

Step 3: Final Merge

[2 4 5 7] + [1 2 3 6] → [1 2 2 3 4 5 6 7]

Elements from both subarrays are compared one by one using two pointers until the final sorted array is obtained.



13 Explain how searching works in a linked list.

A **linked list** is a linear data structure in which elements (nodes) are connected using pointers.

Each node contains **two parts**:

1. **Data**
2. **Link (address of the next node)**

Unlike arrays, linked lists do **not support direct access** using index. Therefore, searching in a linked list is done using **linear search**.

Searching Process in a Linked List

The searching operation follows these steps:



Step 1

Start from the **first node (head)** of the linked list.

Step 2

Compare the **data of the current node** with the element to be searched (key).

Step 3

- If the data matches the key, the search is **successful** and the position is returned.
- If the data does not match, move to the **next node** using the link.

Step 4

Repeat the comparison for each node until:

- The element is found, or
- The end of the linked list is reached (NULL).

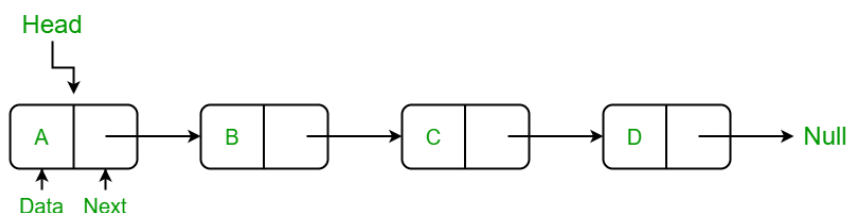
Step 5

If the end is reached and the element is not found, the search is **unsuccessful**.

14.What is a Linked List? Explain features, advantages.

A Linked List is a linear data structure in which elements are stored in the form of nodes. Each node contains two parts:

1. Data part – stores the actual value
2. Link part – stores the address of the next node



Unlike arrays, linked list elements are not stored in consecutive memory locations. Nodes are connected using pointers, which allows dynamic memory allocation.

Features of Linked List

1. A linked list consists of a collection of nodes.



2. Each node contains data and a pointer to the next node.
3. Memory allocation is done dynamically at runtime.
4. Nodes are stored in non-contiguous memory locations.
5. The head node stores the address of the first node.
6. The last node contains NULL in its link part.
7. Elements are accessed sequentially, not randomly.

Advantages of Linked List

1. *Dynamic Size*
The size of a linked list can grow or shrink as needed.
2. *Efficient Insertion and Deletion*
Elements can be inserted or deleted without shifting other elements.
3. *Better Memory Utilization*
Memory is allocated only when required, reducing wastage.
4. *Flexibility*
Linked lists are useful in implementing stacks, queues, graphs, and trees.
5. *No Fixed Size Limitation*
Unlike arrays, linked lists do not require a predefined size.

15. Describe structure of a node and explain how to insert into a singly linked list.

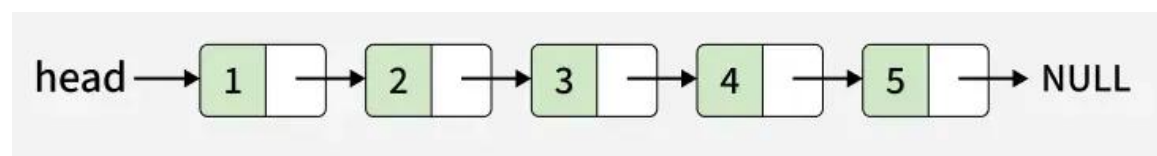
Structure of a Node:

In a singly linked list, each element is represented by a node. A node is a self-referential structure consisting of two fields:

1. Data Field
 - Stores the actual information or value of the node.
 - It may be an integer, character, or any data type.
2. Link Field (Next Pointer)
 - Stores the address of the next node in the linked list.
 - It connects the current node to the next node.

The last node of the linked list has its Next field set to NULL, which indicates the end of the list.

The address of the first node is stored in a pointer called Head.



Singly Linked List Representation



- Nodes are stored in non-contiguous memory locations
- Nodes are connected using pointers

Insertion into a Singly Linked List: Insertion is the process of adding a new node into the linked list.

In a singly linked list, insertion can be performed at three different positions:

1. Insertion at the beginning
2. Insertion at the end
3. Insertion at a specific position (middle)

1. Insertion at the Beginning

In this method, a new node is inserted before the first node.

Steps:

1. Create a new node.
2. Store the data in the new node.
3. Make the new node's Next point to the current head.
4. Update the head to point to the new node.

Result:

The new node becomes the first node of the list.

2. Insertion at the End

In this method, a new node is inserted after the last node.

Steps:

1. Create a new node.
2. Store the data in the new node and set Next = NULL.
3. Traverse the linked list until the last node is reached.
4. Make the last node's Next point to the new node.

Result:

The new node becomes the last node of the list.

3. Insertion at a Specific Position (Middle)

In this method, a new node is inserted at a given position in the list.

Steps:



-
1. Create a new node and store the data.
 2. Traverse the list to the node after which insertion is required.
 3. Set the new node's Next to the next node.
 4. Update the previous node's Next to the new node.

Result:

The new node is inserted at the required position.

Advantages of Insertion in Linked List

- No shifting of elements is required
 - Memory allocation is dynamic
 - Efficient compared to array insertion
-